(19)中华人民共和国国家知识产权局

(12)发明专利申请

(10)申请公布号 CN 109144963 A
(43)申请公布日 2019.01.04

(21)申请号 201710492419.7

(22)申请日 2017.06.26

(71)申请人 阿里巴巴集团控股有限公司
地址 英属开曼群岛大开曼资本大厦一座四层847号邮箱

(72)发明人 周正中

(74)专利代理机构 上海百一领御专利代理事务所(普通合伙) 31243
代理人 陈贞健 王路丰

(51)Int.Cl.
G06F 16/18(2019.01)
G06F 16/13(2019.01)
G06F 16/17(2019.01)

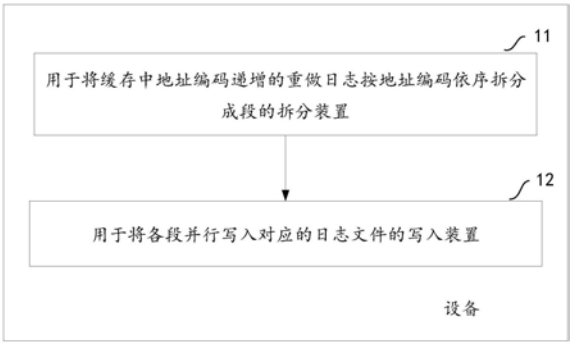
权利要求书2页 说明书18页 附图4页

(54)发明名称

一种重做日志持久化方法及设备

(57)摘要

本申请的目的是提供一种重做日志持久化方法及设备，本申请通过将缓存中地址编码递增的重做日志按照地址编码依序拆分成段，并将各段并行写入对应的日志文件，其中，日志文件对应与不同的磁盘，实现了将缓存中的重做日志并行写入不同磁盘中对应的日志文件，使得磁盘持久化重做日志时的总的数据量与写入缓存的重做日志的数据量匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题，又由于所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增，以便后续从磁盘中按序将重做日志恢复出来。



CN 109144963 A

1.一种重做日志持久化方法,其中,所述方法包括:

将缓存中地址编码递增的重做日志按地址编码依序拆分成段,其中,所述段中的重做日志的地址编码依序保持递增,序号递增的段中的重做日志的地址编码依序保持递增;

将各段并行写入对应的日志文件,其中,所述日志文件对应于不同的磁盘。

2.根据权利要求1所述的方法,其中,将缓存中地址编码递增的重做日志按地址编码依序拆分成段,包括:

每当缓存中地址编码递增的重做日志的数据量达到预设固定值,将所述数据量达到预设固定值的重做日志按地址编码依序拆分成段。

3.根据权利要求2所述的方法,其中,所述按地址编码依序拆分成的段为数据量相同大小的段。

4.根据权利要求2所述的方法,其中,所述按地址编码依序拆分成的段数据量大小,根据对应的待写入的日志文件的预设一次写入数据量限值确定。

5.根据权利要求1所述的方法,其中,将缓存中地址编码递增的重做日志按地址编码依序拆分成段,包括:

将缓存中地址编码递增的重做日志按地址编码依序按预设比例拆分成段,其中,所述预设比例对应于各待写入的日志文件。

6.根据权利要求5所述的方法,其中,将各段并行写入对应的日志文件的并行,将各段中的首尾地址编码写入对应的日志文件。

7.根据权利要求5所述的方法,其中,将各段并行写入对应的日志文件的并行,将按预设比例拆分成各段的总数据量写入每个日志文件。

8.根据权利要求1至7任一项所述的方法,其中,将各段并行写入对应的日志文件包括:

将各段并行写入对应的日志文件;

对完成写入的日志文件个数进行计数;

当所述计数的个数等于日志文件的总个数,则判断各段写入对应的日志文件完成。

9.一种数据持久化方法,其中,所述方法包括:

将缓存中地址编码连续的数据依序拆分成段,其中,所述段中的数据的地址编码保持连续,序号连续的段中的数据的地址编码保持连续;

将各段并行写入对应的数据文件,其中,所述数据文件对应于不同的磁盘。

10.根据权利要求9所述的方法,其中,所述将缓存中地址编码连续的数据依序拆分成段,包括:

每当缓存中地址编码连续的数据的数据量达到预设固定值,将所述数据量达到预设固定值的数据按地址编码依序拆分成段。

11.根据权利要求9所述的方法,其中,将各段并行写入对应的数据文件,包括:

将各段并行写入对应的数据文件;

对完成写入的数据文件个数进行计数;

当所述计数的个数等于数据文件的总个数,则判断各段写入对应的数据文件完成。

12.一种重做日志持久化设备,其中,所述设备包括:

拆分装置,用于将缓存中地址编码递增的重做日志按地址编码依序拆分成段,其中,所述段中的重做日志的地址编码依序保持递增,序号递增的段中的重做日志的地址编码依序

保持递增；

写入装置，用于将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

13.一种基于计算的设备，其中，该设备包括：

处理器；以及

被安排成存储计算机可执行指令的存储器，所述可执行指令在被执行时使所述处理器：

将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

14.一种存储可执行指令的非暂态计算机可读存储介质，在所述可执行指令由电子设备执行时，使得所述电子设备：

将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

一种重做日志持久化方法及设备

技术领域

[0001] 本申请涉及计算机领域,尤其涉及一种重做日志持久化方法及设备。

背景技术

[0002] 数据库为了满足事务持久化的需求(即事务成功的结束后,事务的状态将在数据库中永久的保存),同时为了降低离散IO(Input and Output,输入输出)的性能影响,数据库领域通过重做日志来记录事务产生的日志。

[0003] 其中,重做日志是串行、序列化的,例如一家超市中,有很多顾客在此消费,每一笔消费记录都要按照消费的时间先后顺序,串行地被记录下来,这样每笔消费记录都有一个一直递增的序列来标识该笔消费记录。因此重做日志是有地址编码的,该地址编码也是一直递增。数据库在写重做日志时,首先写入重做日志的缓存区,然后由redo writer(后台进程)根据日志调度,将缓存区中的重做日志持久化到磁盘,并更新最后的持久化地址编码。当数据库中的事务在结束时,需要比对持久化后的地址编码,确保事务产生的重做日志已持久化,如果没有持久化,负责和用户交互的数据库进程会通知redo writer或自己开启持久化任务,并等待持久化结束。例如,用户在往购物篮放商品时,会产生重做日志,产生多少数据量的重做日志,就需要从重做日志对应的地址编码中的上一个地址编码位点起,申请一段没有使用过的地址进行编码(例如申请一段地址编码的长度等于要写入的重做日志的数据量),然后将需要写入的重做日志写入重做日志缓存区,在事务结束时(比如购物买单),也会申请一段重做日志的地址编码,然后将代表该事务结束的重做日志写入重做日志缓存区,假设它的地址编码结束位置为A;在比对的过程,每次从重做日志缓存区将重做日志写入重做日志文件时,都会记录下来已写到哪个重做日志的地址编码了,故为了保证事务持久化,事务结束时,用来代表事务结束的重做日志的地址编码结束位置A必须小于或等于已持久化的重做日志的地址编码,等于的时候说明事务持久化结束,而小于的时候说明事务持久化结束且进入了下一事务的重做日志的持久化。

[0004] 但是,现有技术中,如图1所示,当数据库在写重做日志时,由于缓存区(内存)的性能很高,能将重做日志快速地写入缓存区(redo buffer),但当通过后台进程(redo writer)将缓存区中的重做日志持久化到磁盘中的重做日志文件(redo file)时,即使使用的是目前非常高端的SSD(Solid State Drives,固态硬盘),也比缓存区差10倍以上的持久化性能,故由于缓存区与单块磁盘的性能悬殊很大,导致虽然将重做日志写入缓存区速度很快,但是从缓存区将重做日志持久化至磁盘却成为数据传输的瓶颈。例如将重做日志写入缓存区时为30GB/s,而将重做日志从缓存区持久化到单块磁盘却只有2GB/s,导致缓存区与磁盘写入重做日志速度不对等,进而导致重做日志持久化时数据传输瓶颈。

发明内容

[0005] 本申请的一个目的是提供一种重做日志持久化方法及设备,解决现有技术中数据库写重做日志时的缓存与单块磁盘之间的性能不对等的问题。

[0006] 根据本申请的一个方面，提供了一种重做日志持久化方法，该方法包括：

[0007] 将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

[0008] 将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

[0009] 进一步地，上述方法中，将缓存中地址编码递增的重做日志按地址编码依序拆分成段，包括：

[0010] 每当缓存中地址编码递增的重做日志的数据量达到预设固定值，将所述数据量达到预设固定值的重做日志按地址编码依序拆分成段。

[0011] 进一步地，上述方法中，所述按地址编码依序拆分成的段为数据量相同大小的段。

[0012] 进一步地，上述方法中，所述按地址编码依序拆分成的段数据量大小，根据对应的待写入的日志文件的预设一次写入数据量限值确定。

[0013] 进一步地，上述方法中，所述将缓存中地址编码递增的重做日志按地址编码依序拆分成段，包括：

[0014] 将缓存中地址编码递增的重做日志按地址编码依序按预设比例拆分成段，其中，所述预设比例对应于各待写入的日志文件。

[0015] 进一步地，上述方法中，所述将各段并行写入对应的日志文件的并行，将各段中的首尾地址编码写入对应的日志文件。

[0016] 进一步地，上述方法中，所述将各段并行写入对应的日志文件的并行，将按预设比例拆分成各段的总数据量写入每个日志文件。

[0017] 进一步地，上述方法中，所述将各段并行写入对应的日志文件包括：

[0018] 将各段并行写入对应的日志文件；

[0019] 对完成写入的日志文件个数进行计数；

[0020] 当所述计数的个数等于日志文件的总个数，则判断各段写入对应的日志文件完成。

[0021] 根据本申请的另一个方面，提供了一种数据持久化方法，其中，所述方法包括：

[0022] 将缓存中地址编码连续的数据依序拆分成段，其中，所述段中的数据的地址编码保持连续，序号连续的段中的数据的地址编码保持连续；

[0023] 将各段并行写入对应的数据文件，其中，所述数据文件对应于不同的磁盘。

[0024] 进一步地，上述方法中，所述将缓存中地址编码连续的数据依序拆分成段，包括：

[0025] 每当缓存中地址编码连续的数据的数据量达到预设固定值，将所述数据量达到预设固定值的数据按地址编码依序拆分成段。

[0026] 进一步地，上述方法中，将各段并行写入对应的数据文件，包括：

[0027] 将各段并行写入对应的数据文件；

[0028] 对完成写入的数据文件个数进行计数；

[0029] 当所述计数的个数等于数据文件的总个数，则判断各段写入对应的数据文件完成。

[0030] 根据本申请的另一方面，还提供了一种重做日志持久化设备，该设备包括：

[0031] 拆分装置，用于将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其

中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

[0032] 写入装置，用于将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

[0033] 根据本申请的另一个方面，提供了一种基于计算的设备，其中，该设备包括：

[0034] 处理器；以及

[0035] 被安排成存储计算机可执行指令的存储器，所述可执行指令在被执行时使所述处理器：

[0036] 将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

[0037] 将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

[0038] 根据本申请的另一个方面，提供了一种存储可执行指令的非暂态计算机可读存储介质，在所述可执行指令由电子设备执行时，使得所述电子设备：

[0039] 将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

[0040] 将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

[0041] 与现有技术相比，本申请通过将缓存中地址编码递增的重做日志按照地址编码依序拆分成段，并将各段并行写入对应的日志文件，其中，日志文件对应与不同的磁盘，实现了将缓存中的重做日志并行写入不同磁盘中对应的日志文件，使得磁盘持久化重做日志时的总的数据量与写入缓存的重做日志的数据量匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题，又由于所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增，以便后续从磁盘中按序将重做日志恢复出来。

附图说明

[0042] 通过阅读参照以下附图所作的对非限制性实施例所作的详细描述，本申请的其它特征、目的和优点将会变得更明显：

[0043] 图1示出现有方案的数据库中的缓存与单块磁盘之间的持久化示意图；

[0044] 图2示出根据本申请一个方面的一种在缓存与多个单块磁盘之间的重做日志持久化方法的示意图；

[0045] 图3示出根据本申请一个方面的一个实施例中的一种重做日志持久化方法的示意图；

[0046] 图4示出根据本申请一个方面的另一个实施例中的一种重做日志持久化方法的示意图；

[0047] 图5示出根据本申请一个方面的另一个实施例中的一种重做日志持久化方法的示意图；

[0048] 图6示出根据本申请一个方面的一种重做日志持久化设备的结构示意图。

[0049] 附图中相同或相似的附图标记代表相同或相似的部件。

具体实施方式

[0050] 下面结合附图对本申请作进一步详细描述。

[0051] 在本申请一个典型的配置中，终端、服务网络的设备和可信方均包括一个或多个处理器(CPU)、输入/输出接口、网络接口和内存。

[0052] 内存可能包括计算机可读介质中的非永久性存储器，随机存取存储器(RAM)和/或非易失性内存等形式，如只读存储器(ROM)或闪存(flash RAM)。内存是计算机可读介质的示例。

[0053] 计算机可读介质包括永久性和非永久性、可移动和非可移动媒体可以由任何方法或技术来实现信息存储。信息可以是计算机可读指令、数据结构、程序的模块或其他数据。计算机的存储介质的例子包括，但不限于相变内存(PRAM)、静态随机存取存储器(SRAM)、动态随机存取存储器(DRAM)、其他类型的随机存取存储器(RAM)、只读存储器(ROM)、电可擦除可编程只读存储器(EEPROM)、快闪记忆体或其他内存技术、只读光盘只读存储器(CD-ROM)、数字多功能光盘(DVD)或其他光学存储、磁盒式磁带，磁带磁盘存储或其他磁性存储设备或任何其他非传输介质，可用于存储可以被计算设备访问的信息。按照本文中的界定，计算机可读介质不包括非暂存电脑可读媒体(transitory media)，如调制的数据信号和载波。

[0054] 本申请提供了一种重做日志持久化方法，应用于数据库中的将缓存中的重做日志持久化至磁盘中的场景中，该方法包括：步骤S11和步骤S12，其中，所述步骤S11将缓存中地址编码递增的重做日志按照地址编码依序拆分成段，接着，所述步骤S12将各段并行写入对应的日志文件，其中，日志文件对应与不同的磁盘，实现了将缓存中的重做日志并行写入不同磁盘中对应的日志文件，使得磁盘持久化重做日志时的总的数据量与写入缓存的重做日志的数据量匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题，又由于所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增，以便后续从磁盘中按序将重做日志恢复出来。

[0055] 本申请一实施例中，所述重做日志是按照地址编码依序保持递增的，故可以将缓存(redo buffer)需要持久化至磁盘中的重做日志按照对应的地址编码进行分段，每一段重做日志对应一个日志文件，即每一段重做日志对应地持久化至一个日志文件(redo file)中，使用异步IO(Input and Output输入输出)将缓存中的各段重做日志持久化写入至对应的日志文件中，其中所述日志文件对应于不同的磁盘，进而发挥多块磁盘的持久化重做日志的带宽，抹平缓存与单块磁盘之间存在的带宽和IO瓶颈的问题。

[0056] 本申请一实施例中，如图2所示，若数据库写入缓存中的重做日志的速度为30GB/s，且写入缓存的重做日志的数据量为30GB时，根据需要待持久化的重做日志的数据量及日志文件数量，决定需要并行的持久化后台进程的数量，若本实施例中需要并行的后台进程为15个，且每个后台进程对应每次写入各个日志文件的速度为2GB/s，则通过并行15个后台进程，将缓存中的重做日志的数据量并行地写入至各个日志文件中，例如并行地从通过后台进程1将缓存中的按照地址编码的0-2GB的重做日志写入对应的日志文件1，并行地通过后台进程2将缓存中的按照地址编码的2GB-4GB的重做日志写入对应的日志文件2，依此按照地址编码依序递增的顺序，将缓存中的所有的重做日志并行地写入各个日志文件，使得磁盘持久化重做日志的总的数据量与缓存写重做日志的数据量匹配得上，进而抹平了磁盘

与缓存之间的性能不对等问题。

[0057] 进一步地，所述步骤S12中的将缓存中地址编码递增的重做日志按地址编码依序拆分成段，包括：

[0058] 每当缓存中地址编码递增的重做日志的数据量达到预设固定值，将所述数据量达到预设固定值的重做日志按地址编码依序拆分成段。

[0059] 需要说明的是，所述预设固定值指通过所有的后台进程单次从缓存(redo buffer)中将重做日志对应的持久化写入至各个磁盘中的日志文件(redo file)的重做日志的数据总量。

[0060] 本申请一实施例中，每当缓存中地址编码递增的重做日志的数据量达到预设固定值(假设为128KB)时，将依序保持递增的地址编码所对应的重做日志的所述数据量按照地址编码依序进行拆分成段，得到每段的重做日志为8KB，由于并行的日志文件为16个，则日志文件1从地址编码0开始，将第一段重做日志的数据量8KB持久化写入磁盘1中的日志文件1中，日志文件2从地址编码8KB开始，将第一段重做日志的数据量8KB持久化写入磁盘2中的日志文件2中，依照地址编码依序递增的顺序，将缓存中的所述数据量达到预设固定值的所有的重做日志，通过并发的16个后台进程，并行从缓存中分别持久化写入至对应的日志文件中，即所有并发的日志文件单次可以写入的数据量为16*8KB＝128KB，完成了按照重做日志的地址编码依序递增的顺序，将缓存中的重做日志按段的并发地写入至不同的日志文件，减少了重做日志持久化至磁盘的延迟，还抹平了缓存与单块磁盘之间的重做日志单次写入的数据量的不对等的问题。

[0061] 进一步地，所述按地址编码依序拆分成的段为数据量相同大小的段。下面以每个日志文件单次传输的按照地址编码依序拆分成的段为重做日志的数据量相同大小的段进行说明。即缓存中写满了一批需要持久化至磁盘的重做日志，之后按照地址编码依序将写满的这一批需要持久化至磁盘的重做日志进行分段，得到的每段重做日志对应地址编码的固定的相同数据量大小，例如为BATCH_SIZE，即每次分别并行地向各个对应的日志文件(redo file)写入的重做日志的数据量为BATCH_SIZE，为每个日志文件进行依序递增的编号，从日志文件1开始编号，假设缓存中已写满的一批需要持久化至磁盘的重做日志需要有N个日志文件，则日志文件编号分别为日志文件1、日志文件2、……、日志文件N，其中每段重做日志(BATCH_SIZE)中包括按照地址编码依序保持递增的至少一个重做日志，每个重做日志的数据量为(REDO_BLOCK_SIZE，重做日志的最小组成单位)，即例如每段重做日志(BATCH_SIZE)＝M*一个重做日志(REDO_BLOCK_SIZE)，其中M大于等于1的正整数，例如REDO_BLOCK_SIZE＝8KB等。

[0062] 本申请一实施例中，若缓存中已写满了一批需要持久化至磁盘的重做日志需要N个日志文件进行持久化，则每个日志文件对应的重做日志写入的地址编码的计算方法如下：

[0063] 将第一段重做日志对应的持久化写入1号日志文件的地址编码中的起点地址编码为s_pos＝mod(address,(BATCH_SIZE*N))＝0，终点地址编码为s_pos+BATCH_SIZE，即一次写入日志文件1的该段重做日志的数据量大小为BATCH_SIZE；

[0064] 将第二段重做日志对应的持久化写入2号日志文件的地址编码中的起点地址编码为s_pos＝mod(address,(BATCH_SIZE*N))＝BATCH_SIZE*1，终点地址编码为s_pos+BATCH_

SIZE；

[0065] 以此类推，……；

[0066] 将第n段重做日志对应的持久化写入n号日志文件的地址编码中的起点地址编码s_pos＝mod(address,(BATCH_SIZE*N))＝BATCH_SIZE*(N-1)，终点为s_pos+BATCH_SIZE。

[0067] 当然，上述将各段重做日志写入对应的日志文件时，是通过后台进程(redo writer)进程并行写入的，使得磁盘持久化重做日志的总的数据量(并行写入N个日志文件的总的数据量)与缓存写重做日志的数据量(即缓存中已写满了一批需要持久化至磁盘的重做日志的数据量)匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题。

[0068] 例如，如图3所示，缓存中已写入了一批需要持久化至磁盘的重做日志为256KB，若按照依序保持递增的地址编码对应的重做日志拆分成段，每段重做日志为16KB，即每段里面有2个重做日志，则有第一段中的2个重做日志的地址编码分别为0、8KB，第二段中的2个重做日志的地址编码分别为16KB、24KB，依此类推，得到第16段中的2个重做日志的地址编码分别为240KB、248KB；根据重做日志中的地址编码及数据库配置，得到可以并行进行持久化写入重做日志至磁盘的日志文件为2个，日志文件1和日志文件2，则并行地将第一段重做日志持久化写入日志文件1，并行将第二段重做日志并行地对应地写入至日志文件2，日志文件1写完第一段重做日志之后，继续并行持久化写入第三段重做日志，日志文件2写完第二段重做日志之后，继续并行持久化写入第四段重做日志，依此并行持久化过程通过2个并行地后台进程(redo writer)将所有的16段重做日志均持久化写入至对应的日志文件之后，用于指示将需要从缓存已写入的一批需要持久化至磁盘的重做日志(256KB)完成写入至各个磁盘中，进而指示此次持久化写入完成。

[0069] 在进行数据库恢复时，按照序号递增的段中的重做日志的地址编码依序保持递增的顺序，由于每段重做日志对应的数据量大小相同，故按照各段的序号依序递增，即第一段、第二段、……、第16段，将对应持久化写入至日志文件1中的地址编码分别为0、8KB，日志文件中的地址编码分别为16KB、24KB，依此类推，按照序号递增的段中的重做日志的地址编码依序保持递增的顺序，将所有持久化写入至各个磁盘中的日志文件中的重做日志进行合并，恢复出一份完整的有序的持久化至各磁盘中的重做日志。

[0070] 进一步地，所述按地址编码依序拆分成的段数据量大小，根据对应的待写入的日志文件的预设一次写入数据量限值确定。下面以每个日志文件单次传输的按照地址编码依序拆分成的段数据量的大小为根据对应写入的日志文件的预设一次写入数据量限值来确定进行说明。即缓存中写满了一批需要持久化至磁盘的重做日志，确定每个日志文件依稀写入数据量限值，之后按照地址编码依序将写满的这一批需要持久化至磁盘的重做日志进行分段，得到对应各个日志文件的预设一次写入数据量限值对应的每段重做日志，即每个日志文件单次持久化写入的重做日志的数据量不同。例如，每个日志文件存放在不同的磁盘中，由于不同的磁盘，其对应的写入性能可能存在不一致，故为每个磁盘中的日志文件预设一次写入数据量限值，达到在持久化写入重做日志时满足不同的磁盘的写入性能。

[0071] 本申请一实施例中，假设缓存中已写满的一批需要持久化至磁盘的重做日志需要有N个日志文件，则日志文件编号分别为日志文件1、日志文件2、……、日志文件N，其中每段重做日志(BATCH_SIZE)中包括按照地址编码依序保持递增的至少一个重做日志。所述预设一次写入数据量限值采用LOCAL_BATCH_SIZE进行表示，则有缓存中已写满的一批需要持久

化至磁盘的重做日志的中的数据量为SUM_BATCH_SIZE,按照重做日志中的地址编码对一批需要持久化至磁盘的重做日志的中的数据量进行分段,得到对应的n段重做日志,若缓存中已写满的一批需要持久化至磁盘的重做日志需要N个日志文件进行并行持久化,则每个日志文件对应的重做日志写入的地址编码的计算方法如下:

[0072] 将第一段重做日志对应的持久化写入1号日志文件的地址编码中的起点地址编码为s_pos=mod(address,(SUM_BATCH_SIZE))=0,终点地址编码为s_pos+LOCAL_BATCH_SIZE1,即一次写入日志文件1的该段重做日志的数据量限值为大小为LOCAL_BATCH_SIZE1;

[0073] 将第二段重做日志对应的持久化写入2号日志文件的地址编码中的起点地址编码为:

[0074] s_pos=mod(address,(SUM_BATCH_SIZE))=LOCAL_BATCH_SIZE1,终点地址编码为s_pos+LOCAL_BATCH_SIZE2,即一次写入日志文件2的该段重做日志的数据量限值为大小为LOCAL_BATCH_SIZE2;

[0075] 以此类推,……;

[0076] 将第n段重做日志对应的持久化写入n号日志文件的地址编码中的起点地址编码为:

[0077] s_pos=mod(address,(SUM_BATCH_SIZE))=LOCAL_BATCH_SIZE(n-1)的位置,终点地址编码为s_pos+LOCAL_BATCH_SIZEn。

[0078] 当然,上述将各段重做日志写入对应的日志文件时,是通过后台进程(redo writer)进程并行写入的,使得磁盘持久化重做日志的总的数据量(并行写入N个日志文件的总的数据量)与缓存写重做日志的数据量(即缓存中已写满了一批需要持久化至磁盘的重做日志的数据量)匹配得上,进而抹平了磁盘与缓存之间的性能不对等问题。

[0079] 例如,如图4所示,缓存中已写入了一批需要持久化至磁盘的重做日志为288KB,根据重做日志中的地址编码及数据库配置,得到可以并行进行持久化写入重做日志至磁盘的日志文件为2个,日志文件1和日志文件2;若按照依序保持递增的地址编码对应的重做日志拆分成段,日志文件1对应待持久化的对应段重做日志为16KB,即日志文件1预设一次写入数据量限值为16KB,日志文件2对应待持久化的对应段重做日志为32KB,即日志文件2预设一次写入数据量限值为32KB,则并行地将第一段对应的16KB重做日志持久化写入日志文件1,同时将第二段对应的32KB重做日志并行地对应地写入至日志文件2,当日志文件1写完第一段对应的16KB重做日志之后,继续并行持久化写入第三段对应的16KB重做日志,当日志文件2写完第二段对应的32KB重做日志之后,继续并行持久化写入第四段对应的32KB重做日志,依此并行持久化过程通过2个并行地后台进程(redo writer)将所有的12段重做日志均持久化写入至对应的日志文件之后,用于指示将需要从缓存已写入的一批需要持久化至磁盘的重做日志(288KB)完成写入至各个磁盘中,进而不仅在考虑到不同的磁盘的写入性能的同时,还将缓存中已写入的一批需要持久化至磁盘的重做日志(288KB)按照地址编码持久化写入完成。

[0080] 在进行数据库恢复时,从重做日志的地址编码起点0开始,按照日志文件1与日志文件2分别的预设一次写入数据量限值和地址编码依序保持递增的重做日志,即16KB和32KB,从各个序号递增的第一段、第二段、……、第6段对应写入的日志文件中将重做日志恢复出来。例如将日志文件1中的从地址编码为0开始的第一段对应的16KB重做日志、日志文

件2中的从地址编码为16KB开始的第二段对应的32KB重做日志、日志文件1中的从地址编码为48KB开始的第三段对应的16KB重做日志、日志文件2中的从地址编码为64KB开始的第四段对应的32KB重做日志、……、日志文件1中的从地址编码为240KB开始的第11段对应的16KB重做日志、日志文件2中的从地址编码为256KB开始的第12段对应的32KB重做日志进行恢复，即，按照序号递增的段中的重做日志的地址编码依序保持递增的顺序，将所有持久化写入至各个磁盘中的日志文件中的重做日志进行合并，恢复出一份完整的有序的持久化至各磁盘中的重做日志。

[0081] 进一步地，所述步骤S11中的将缓存中地址编码递增的重做日志按地址编码依序拆分成段，包括：

[0082] 将缓存中地址编码递增的重做日志按地址编码依序按预设比例拆分成段，其中，所述预设比例对应于各待写入的日志文件。

[0083] 在实际的数据库应用场景中，上述图3对应的按地址编码依序拆分成的段为数据量相同大小的段和图4对应的所述按地址编码依序拆分成的段数据量大小，根据对应的待写入的日志文件的预设一次写入数据量限值确定需要在缓存中已经写入完整的一批需要持久化至磁盘的重做日志，才进行拆分成段，而此处将缓存中地址编码递增的重做日志按地址编码依序按预设比例拆分成段的持久化方法，即使缓存中写入的重做日志的数据量很少时，也是可以按照预设比例进行并行得持久化的，进而降低了将缓存中的重做日志持久化写入至磁盘等待时间(时延)，按照预设比例进行拆分成段并行地持久化写入各个日志文件中的总的数据量的大小可大可小，以满足更及时地将缓存中写入的重做日志持久化写入至各磁盘中的日志文件中。。

[0084] 本申请一实施例中，由于缓存中的重做日志产生的数量和速度与数据库的写入请求负载有很大关系，写入请求越密集，产生的重做日志就越多；写入请求约少，产生的重做日志就越少。上述对缓存中写入的重做日志按照预设比例拆分成段的并行持久化方法，可以很好的匹配不同的负载，当缓存中的重做日志的数据很少时，也可以拆分成多段分别并行地写入至日志文件中，在重做日志的数据量很少的时减少了持久化写入的等待时间，同时也降低吞吐量(因为每一次数据库中的写请求都有额外的开销，比如磁头寻道时间)，由于该吞吐量是弹性的，在写入重做日志的数据量较少时对吞吐量是没有要求了，即使吞吐量降低了，也不会影响到按照预设比例进行并行持久化至磁盘的持久化进程，还降低了持久化写入的等待时间(时延)；当缓存写满一批重做日志的数据量大时，通过后台进程并行地每次写入单个磁盘的重做日志的数据量也相应增大，吞吐量就更大。

[0085] 本申请一实施例中，如图5所示，基于重做日志中的地址编码与数据库配置，确定需要并行持久化的日志文件为4个，分别为不同的后台进程对应的日志文件1、日志文件2、日志文件3和日志文件4。若缓存中写入了200KB的重做日志，则基于预设比例将写入的这200KB的重做日志拆分成4段。例如预设比例为1:2:3:4，那么所述预设比例1:2:3:4分别对应于待写入重做日志的日志文件1、日志文件2、日志文件3及日志文件4，则有将第一段对应的20KB的重做日志通过后台进程写入日志文件1中，将第二段对应的40KB的重做日志通过后台进程写入日志文件2中，将第三段对应的60KB的重做日志通过后台进程写入日志文件3中，将第四段对应的80KB的重做日志通过后台进程写入日志文件4中，当然将上述四段重做日志写入对应的日志文件时，是并行持久化进行的，实现了将缓存中写入的一批重做日志

按照地址编码依序保持递增的顺序分别写入个日志文件。若缓存中写入了100KB的重做日志，则基于预设比例继续将写入的这100KB的重做日志拆分成4段。例如预设比例为1:2:3:4，那么所述预设比例1:2:3:4分别对应于待写入重做日志的日志文件1、日志文件2、日志文件3及日志文件4，则有将第一段对应的10KB的重做日志通过后台进程写入日志文件1中，将第二段对应的20KB的重做日志通过后台进程写入日志文件2中，将第三段对应的30KB的重做日志通过后台进程写入日志文件3中，将第四段对应的40KB的重做日志通过后台进程写入日志文件4中，当然将上述四段重做日志写入对应的日志文件时，是并行持久化进行的，实现了将缓存中写入的一批重做日志按照地址编码依序保持递增的顺序分别写入个日志文件。

[0086] 为了将每次按照预设比例拆分成段并分别并行持久化写入对应的日志文件中的重做日志进行恢复，则所述步骤S12将各段并行写入对应的日志文件的同时，还将各段中的首尾地址编码写入对应的日志文件。例如，由于重做日志的地址编码都是该重做日志的开始地址，故假设缓存中写入了一批重做日志为80KB，按照预设比例(例如1:2:3:4)将该批重做日志进行分段，得到四段重做日志，假设第一段对应的重做日志包括1个重做日，起点地址编码为0，终点地址编码为8KB，第一段对应的重做日志包括2个重做日志，起点地址编码为8KB，终点地址编码为24KB，第三段对应的重做日志包括3个重做日志，起点地址编码为24，终点地址编码为48KB，第四段对应的重做日志包括4个重做日志，起点地址编码为48，终点地址编码为100KB，为了保证后续将持久化写入各个日志文件的重做日志恢复为一份完成有序地重做日志，则在将第一段对应的8KB重做日志写入日志文件1的并行，还将第一段中的首尾地址编码[0,8KB]写入对应的日志文件1中，依此类推，按照预设比例，将各段对应的重做日志分别并行地持久化写入对应的日志文件的同时，还将每段中对应的重做日志的地址编码中的首尾地址编码一并写入对应的日志文件，以便后续基于各段中的首尾地址编码将所有的日志文件中的重做日志恢复出来。

[0087] 为了将每次按照预设比例拆分成段并分别并行持久化写入对应的日志文件中的重做日志进行恢复，则所述步骤S12将各段并行写入对应的日志文件的同时，将按预设比例拆分成各段的总数据量写入每个日志文件。例如，由于重做日志的地址编码都是该重做日志的开始地址，故假设缓存中写入了一批重做日志为80KB，按照预设比例(例如1:2:3:4)将该批重做日志进行分段，得到四段重做日志，假设第一段对应的8KB重做日志包括1个重做日，起点地址编码为0，即第一段的总数据量为8KB，第一段对应的重做日志包括2个重做日志，即第二段的总数据量为16KB，第三段对应的重做日志包括3个重做日志，即第三段的总数据量为24KB，第四段对应的重做日志包括4个重做日志，即第四段的总数据量为32KB，为了保证后续将持久化写入各个日志文件的重做日志恢复为一份完成有序地重做日志，则在将第一段对应的8KB重做日志写入日志文件1的同时，还将第一段中的中数据量8KB写入对应的日志文件1中，依此类推，按照预设比例，将各段对应的重做日志分别并行地持久化写入对应的日志文件的同时，还将每段中对应的重做日志的总数据量一并写入对应的日志文件，以便后续基于各段中的首尾地址编码将所有的日志文件中的重做日志恢复出来。

[0088] 例如，在数据库的恢复过程中，按照序号递增的段中的重做日志的地址编码依序保持递增，分别将写入日志文件1中的第一段对应的8KB重做日志，写入日志文件2中的第二段对应16KB重做日志，写入日志文件3中第三段对应的24KB重做日志，写入日志文件4中的

第四段对应的32KB重做日志依序地合并，恢复出一份完整地有序地重做日志，实现对持久化至不同磁盘中的日志文件中的重做日志的恢复。

[0089] 为了保证完整地将每一批的需要持久化写入重做日志持久化至对应的日志文件中的，所述步骤S12将各段并行写入对应的日志文件包括：

[0090] 将各段并行写入对应的日志文件；

[0091] 对完成写入的日志文件个数进行计数；

[0092] 当所述计数的个数等于日志文件的总个数，则判断各段写入对应的日志文件完成。

[0093] 本申请一实施例中，将缓存中写入的每一批需要持久化的重做日志，并行地写入至日志文件时，根据地址编码与数据库中的系统配置，获取当前这一批需要持久化至日志文件的日志文件个数n，当每段重做日志完成写入对应的日志文件之后，需要对完成写入的日志文件个数进行计数并更新计数，当所述计数的个数达到参与并行持久化写入的日志文件的总个数n时，就判断各段重做日志持久化写入至对应的日志文件完成，即完成了将缓存中写入的这一批需要持久化的重做日志的持久化写入。

[0094] 根据本申请的另一个方面的实施例提供的一种数据持久化方法，应用于数据库中的将缓存中数据持久化至磁盘中的场景中，该方法包括：

[0095] 将缓存中地址编码连续的数据依序拆分成段，其中，所述段中的数据的地址编码保持连续，序号连续的段中的数据的地址编码保持连续；在此，所述地址编码连续的数据可以包括地址编码连续递增的数据，还可以包括地址编码连续递减的数据。将各段并行写入对应的数据文件，其中，所述数据文件对应于不同的磁盘。实现了将缓存中的连续的数据并行写入不同磁盘中对应的数据文件，使得磁盘持久化数据时的总的数据量与写入缓存的数的数据量匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题，又由于所述段中的数据的地址编码依序保持连续，序号连续的各段中的书的地址编码亦依序保持连续，以便后续从磁盘中按序将数据恢复出来。

[0096] 本申请一实施例中，所述将缓存中地址编码连续的数据依序拆分成段，包括：

[0097] 每当缓存中地址编码连续的数据的数据量达到预设固定值，将所述数据量达到预设固定值的数据按地址编码依序拆分成段。

[0098] 需要说明的是，所述预设固定值指通过所有的后台进程单次从缓存(redo buffer)中将数据对应的持久化写入至各个磁盘中的数据文件(data file)的数据的数据总量。

[0099] 本申请一实施例中，每当缓存中地址编码连续的数据的数据量达到预设固定值(假设为128KB)时，将依序保持连续的地址编码所对应的数据的所述数据量按照地址编码依序进行拆分成段，得到每段的数据为8KB，由于并行的数据文件为16个，则数据文件1从地址编码0开始，将第一段数据的数据量8KB持久化写入磁盘1中的数据文件1中，数据文件2从地址编码8KB开始，将第一段数据的数据量8KB持久化写入磁盘2中的数据文件2中，依照地址编码依序连续的顺序，将缓存中的所述数据量达到预设固定值的所有的数据，通过并发的16个后台进程，并行从缓存中分别持久化写入至对应的数据文件中，即所有并发的数据文件单次可以写入的数据量为16*8KB＝128KB，完成了按照数据的地址编码依序连续的顺序，将缓存中的数据按段的并发地写入至不同的数据文件，减少了数据持久化至磁盘的延

迟,还抹平了缓存与单块磁盘之间的数单次写入的数据量的不对等的问题。

[0100] 本申请一实施例中,将各段并行写入对应的数据文件,包括:

[0101] 将各段并行写入对应的数据文件;

[0102] 对完成写入的数据文件个数进行计数;

[0103] 当所述计数的个数等于数据文件的总个数,则判断各段写入对应的数据文件完成。

[0104] 本申请一实施例中,将缓存中写入的每一批需要持久化的数据,并行地写入至数据文件时,根据地址编码与数据库中的系统配置,获取当前这一批需要持久化至数据文件的数据文件个数n,当每段数据完成写入对应的数据文件之后,需要对完成写入的数据文件个数进行计数并更新计数,当所述计数的个数达到参与并行持久化写入的数据文件的总个数n时,就判断各段重做数据持久化写入至对应的数据文件完成,即完成了将缓存中写入的这一批需要持久化的重做数据的持久化写入。

[0105] 图6示出了本申请一个方面提供的一种重做日志持久化设备的结构示意图,应用于数据库中的将缓存中的重做日志持久化至磁盘中的场景中,该设备包括:拆分装置11和写入装置12,其中,所述拆分装置11用于:将缓存中地址编码递增的重做日志按照地址编码依序拆分成段,接着,所述写入装置12用于:将各段并行写入对应的日志文件,其中,日志文件对应与不同的磁盘,实现了将缓存中的重做日志并行写入不同磁盘中对应的日志文件,使得磁盘持久化重做日志时的总的数据量与写入缓存的重做日志的数据量匹配得上,进而抹平了磁盘与缓存之间的性能不对等问题,又由于所述段中的重做日志的地址编码依序保持递增,序号递增的段中的重做日志的地址编码依序保持递增,以便后续从磁盘中按序将重做日志恢复出来。

[0106] 本申请一实施例中,所述重做日志是按照地址编码依序保持递增的,故可以将缓存(redo buffer)需要持久化至磁盘中的重做日志按照对应的地址编码进行分段,每一段重做日志对应一个日志文件,即每一段重做日志对应地持久化至一个日志文件(redo file)中,使用异步IO(Input and Output输入输出)将缓存中的各段重做日志持久化写入至对应的日志文件中,其中所述日志文件对应于不同的磁盘,进而发挥多块磁盘的持久化重做日志的带宽,抹平缓存与单块磁盘之间存在的带宽和IO瓶颈的问题。

[0107] 本申请一实施例中,如图2所示,若数据库写入缓存中的重做日志的速度为30GB/s,且写入缓存的重做日志的数据量为30GB时,根据需要待持久化的重做日志的数据量及日志文件数量,决定需要并行的持久化后台进程的数量,若本实施例中需要并行的后台进程为15个,且每个后台进程对应每次写入各个日志文件的速度为2GB/s,则通过并行15个后台进程,将缓存中的重做日志的数据量并行地写入至各个日志文件中,例如并行地从通过后台进程1将缓存中的按照地址编码的0-2GB的重做日志写入对应的日志文件1,并行地通过后台进程2将缓存中的按照地址编码的2GB-4GB的重做日志写入对应的日志文件2,依此按照地址编码依序递增的顺序,将缓存中的所有的重做日志并行地写入各个日志文件,使得磁盘持久化重做日志的总的数据量与缓存写重做日志的数据量匹配得上,进而抹平了磁盘与缓存之间的性能不对等问题。

[0108] 进一步地,所述写入装置12用于:

[0109] 每当缓存中地址编码递增的重做日志的数据量达到预设固定值,将所述数据量达

到预设固定值的重做日志按地址编码依序拆分成段。

[0110] 需要说明的是，所述预设固定值指通过所有的后台进程单次从缓存(redo buffer)中将重做日志对应的持久化写入至各个磁盘中的日志文件(redo file)的重做日志的数据总量。

[0111] 本申请一实施例中，每当缓存中地址编码递增的重做日志的数据量达到预设固定值(假设为128KB)时，将依序保持递增的地址编码所对应的重做日志的所述数据量按照地址编码依序进行拆分成段，得到每段的重做日志为8KB，由于并行的日志文件为16个，则日志文件1从地址编码0开始，将第一段重做日志的数据量8KB持久化写入磁盘1中的日志文件1中，日志文件2从地址编码8KB开始，将第一段重做日志的数据量8KB持久化写入磁盘2中的日志文件2中，依照地址编码依序递增的顺序，将缓存中的所述数据量达到预设固定值的所有的重做日志，通过并发的16个后台进程，并行从缓存中分别持久化写入至对应的日志文件中，即所有并发的日志文件单次可以写入的数据量为16*8KB＝128KB，完成了按照重做日志的地址编码依序递增的顺序，将缓存中的重做日志按段的并发地写入至不同的日志文件，减少了重做日志持久化至磁盘的延迟，还抹平了缓存与单块磁盘之间的重做日志单次写入的数据量的不对等的问题。

[0112] 进一步地，所述按地址编码依序拆分成的段为数据量相同大小的段。下面以每个日志文件单次传输的按照地址编码依序拆分成的段为重做日志的数据量相同大小的段进行说明。即缓存中写满了一批需要持久化至磁盘的重做日志，之后按照地址编码依序将写满的这一批需要持久化至磁盘的重做日志进行分段，得到的每段重做日志对应地址编码的固定的相同数据量大小，例如为BATCH_SIZE，即每次分别并行地向各个对应的日志文件(redo file)写入的重做日志的数据量为BATCH_SIZE，为每个日志文件进行依序递增的编号，从日志文件1开始编号，假设缓存中已写满的一批需要持久化至磁盘的重做日志需要有N个日志文件，则日志文件编号分别为日志文件1、日志文件2、……、日志文件N，其中每段重做日志(BATCH_SIZE)中包括按照地址编码依序保持递增的至少一个重做日志，每个重做日志的数据量为(REDO_BLOCK_SIZE，重做日志的最小组成单位)，即例如每段重做日志(BATCH_SIZE)＝M*一个重做日志(REDO_BLOCK_SIZE)，其中M大于等于1的正整数，例如REDO_BLOCK_SIZE＝8KB等。

[0113] 本申请一实施例中，若缓存中已写满了一批需要持久化至磁盘的重做日志需要N个日志文件进行持久化，则每个日志文件对应的重做日志写入的地址编码的计算方法如下：

[0114] 将第一段重做日志对应的持久化写入1号日志文件的地址编码中的起点地址编码为s_pos＝mod(address,(BATCH_SIZE*N))＝0，终点地址编码为s_pos+BATCH_SIZE，即一次写入日志文件1的该段重做日志的数据量大小为BATCH_SIZE；

[0115] 将第二段重做日志对应的持久化写入2号日志文件的地址编码中的起点地址编码为s_pos＝mod(address,(BATCH_SIZE*N))＝BATCH_SIZE*1，终点地址编码为s_pos+BATCH_SIZE；

[0116] 以此类推，……；

[0117] 将第n段重做日志对应的持久化写入n号日志文件的地址编码中的起点地址编码s_pos＝mod(address,(BATCH_SIZE*N))＝BATCH_SIZE*(N-1)，终点为s_pos+BATCH_SIZE。

[0118] 当然，上述将各段重做日志写入对应的日志文件时，是通过后台进程(redo writer)进程并行写入的，使得磁盘持久化重做日志的总的数据量(并行写入N个日志文件的总的数据量)与缓存写重做日志的数据量(即缓存中已写满了一批需要持久化至磁盘的重做日志的数据量)匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题。

[0119] 例如，如图3所示，缓存中已写入了一批需要持久化至磁盘的重做日志为256KB，若按照依序保持递增的地址编码对应的重做日志拆分成段，每段重做日志为16KB，即每段里面有2个重做日志，则有第一段中的2个重做日志的地址编码分别为0、8KB，第二段中的2个重做日志的地址编码分别为16KB、24KB，依此类推，得到第16段中的2个重做日志的地址编码分别为240KB、248KB；根据重做日志中的地址编码及数据库配置，得到可以并行进行持久化写入重做日志至磁盘的日志文件为2个，日志文件1和日志文件2，则并行地将第一段重做日志持久化写入日志文件1，并行将第二段重做日志并行地对应地写入至日志文件2，日志文件1写完第一段重做日志之后，继续并行持久化写入第三段重做日志，日志文件2写完第二段重做日志之后，继续并行持久化写入第四段重做日志，依此并行持久化过程通过2个并行地后台进程(redo writer)将所有的16段重做日志均持久化写入至对应的日志文件之后，用于指示将需要从缓存已写入的一批需要持久化至磁盘的重做日志(256KB)完成写入至各个磁盘中，进而指示此次持久化写入完成。

[0120] 在进行数据库恢复时，按照序号递增的段中的重做日志的地址编码依序保持递增的顺序，由于每段重做日志对应的数据量大小相同，故按照各段的序号依序递增，即第一段、第二段、……、第16段，将对应持久化写入至日志文件1中的地址编码分别为0、8KB，日志文件中的地址编码分别为16KB、24KB，依此类推，按照序号递增的段中的重做日志的地址编码依序保持递增的顺序，将所有持久化写入至各个磁盘中的日志文件中的重做日志进行合并，恢复出一份完整的有序的持久化至各磁盘中的重做日志。

[0121] 进一步地，所述按地址编码依序拆分成的段数据量大小，根据对应的待写入的日志文件的预设一次写入数据量限值确定。下面以每个日志文件单次传输的按照地址编码依序拆分成的段数据量的大小为根据对应写入的日志文件的预设一次写入数据量限值来确定进行说明。即缓存中写满了一批需要持久化至磁盘的重做日志，确定每个日志文件依稀写入数据量限值，之后按照地址编码依序将写满的这一批需要持久化至磁盘的重做日志进行分段，得到对应各个日志文件的预设一次写入数据量限值对应的每段重做日志，即每个日志文件单次持久化写入的重做日志的数据量不同。例如，每个日志文件存放在不同的磁盘中，由于不同的磁盘，其对应的写入性能可能存在不一致，故为每个磁盘中的日志文件预设一次写入数据量限值，达到在持久化写入重做日志时满足不同的磁盘的写入性能。

[0122] 本申请一实施例中，假设缓存中已写满的一批需要持久化至磁盘的重做日志需要有N个日志文件，则日志文件编号分别为日志文件1、日志文件2、……、日志文件N，其中每段重做日志(BATCH_SIZE)中包括按照地址编码依序保持递增的至少一个重做日志。所述预设一次写入数据量限值采用LOCAL_BATCH_SIZE进行表示，则有缓存中已写满的一批需要持久化至磁盘的重做日志的中的数据量为SUM_BATCH_SIZE，按照重做日志中的地址编码对一批需要持久化至磁盘的重做日志的中的数据量进行分段，得到对应的n段重做日志，若缓存中已写满的一批需要持久化至磁盘的重做日志需要N个日志文件进行并行持久化，则每个日志文件对应的重做日志写入的地址编码的计算方法如下：

[0123] 将第一段重做日志对应的持久化写入1号日志文件的地址编码中的起点地址编码为s_pos＝mod(address,(SUM_BATCH_SIZE))＝0,终点地址编码为s_pos+LOCAL_BATCH_SIZE1,即一次写入日志文件1的该段重做日志的数据量限值为大小为LOCAL_BATCH_SIZE1;

[0124] 将第二段重做日志对应的持久化写入2号日志文件的地址编码中的起点地址编码为:s_pos＝mod(address,(SUM_BATCH_SIZE))＝LOCAL_BATCH_SIZE1,终点地址编码为:s_pos+LOCAL_BATCH_SIZE2,即一次写入日志文件2的该段重做日志的数据量限值为大小为LOCAL_BATCH_SIZE2;

[0125] 以此类推,……;

[0126] 将第n段重做日志对应的持久化写入n号日志文件的地址编码中的起点地址编码为:s_pos＝mod(address,(SUM_BATCH_SIZE))＝LOCAL_BATCH_SIZE(n-1)的位置,终点地址编码为s_pos+LOCAL_BATCH_SIZEn。

[0127] 当然,上述将各段重做日志写入对应的日志文件时,是通过后台进程(redo writer)进程并行写入的,使得磁盘持久化重做日志的总的数据量(并行写入N个日志文件的总的数据量)与缓存写重做日志的数据量(即缓存中已写满了一批需要持久化至磁盘的重做日志的数据量)匹配得上,进而抹平了磁盘与缓存之间的性能不对等问题。

[0128] 例如,如图4所示,缓存中已写入了一批需要持久化至磁盘的重做日志为288KB,根据重做日志中的地址编码及数据库配置,得到可以并行进行持久化写入重做日志至磁盘的日志文件为2个,日志文件1和日志文件2;若按照依序保持递增的地址编码对应的重做日志拆分成段,日志文件1对应待持久化的对应段重做日志为16KB,即日志文件1预设一次写入数据量限值为16KB,日志文件2对应待持久化的对应段重做日志为32KB,即日志文件2预设一次写入数据量限值为32KB,则并行地将第一段对应的16KB重做日志持久化写入日志文件1,并行将第二段对应的32KB重做日志并行地对应地写入至日志文件2,当日志文件1写完第一段对应的16KB重做日志之后,继续并行持久化写入第三段对应的16KB重做日志,当日志文件2写完第二段对应的32KB重做日志之后,继续并行持久化写入第四段对应的32KB重做日志,依此并行持久化过程通过2个并行地后台进程(redo writer)将所有的12段重做日志均持久化写入至对应的日志文件之后,用于指示将需要从缓存已写入的一批需要持久化至磁盘的重做日志(288KB)完成写入至各个磁盘中,进而不仅在考虑到不同的磁盘的写入性能的同时,还将缓存中已写入的一批需要持久化至磁盘的重做日志(288KB)按照地址编码持久化写入完成。

[0129] 在进行数据库恢复时,从重做日志的地址编码起点0开始,按照日志文件1与日志文件2分别的预设一次写入数据量限值和地址编码依序保持递增的重做日志,即16KB和32KB,从各个序号递增的第一段、第二段、……、第6段对应写入的日志文件中将重做日志恢复出来。例如将日志文件1中的从地址编码为0开始的第一段对应的16KB重做日志、日志文件2中的从地址编码为16KB开始的第二段对应的32KB重做日志、日志文件1中的从地址编码为48KB开始的第三段对应的16KB重做日志、日志文件2中的从地址编码为64KB开始的第四段对应的32KB重做日志、……、日志文件1中的从地址编码为240KB开始的第11段对应的16KB重做日志、日志文件2中的从地址编码为256KB开始的第12段对应的32KB重做日志进行恢复,即,按照序号递增的段中的重做日志的地址编码依序保持递增的顺序,将所有持久化写入至各个磁盘中的日志文件中的重做日志进行合并,恢复出一份完整的有序的持久化至

各磁盘中的重做日志。

[0130] 进一步地,所述拆分装置11用于:

[0131] 将缓存中地址编码递增的重做日志按地址编码依序按预设比例拆分成段,其中,所述预设比例对应于各待写入的日志文件。

[0132] 在实际的数据库应用场景中,上述图3对应的按地址编码依序拆分成的段为数据量相同大小的段和图4对应的所述按地址编码依序拆分成的段数据量大小,根据对应的待写入的日志文件的预设一次写入数据量限值确定需要在缓存中已经写入完整的一批需要持久化至磁盘的重做日志,才进行拆分成段,而此处将缓存中地址编码递增的重做日志按地址编码依序按预设比例拆分成段的持久化方法,即使缓存中写入的重做日志的数据量很少时,也是可以按照预设比例进行并行得持久化的,进而降低了将缓存中的重做日志持久化写入至磁盘等待时间(时延),按照预设比例进行拆分成段并行地持久化写入各个日志文件中的总的数据量的大小可大可小,以满足更及时地将缓存中写入的重做日志持久化写入至各磁盘中的日志文件中。。

[0133] 本申请一实施例中,由于缓存中的重做日志产生的数量和速度与数据库的写入请求负载有很大关系,写入请求越密集,产生的重做日志就越多;写入请求约少,产生的重做日志就越少。上述对缓存中写入的重做日志按照预设比例拆分成段的并行持久化方法,可以很好的匹配不同的负载,当缓存中的重做日志的数据很少时,也可以拆分成多段分别并行地写入至日志文件中,在重做日志的数据量很少的时减少了持久化写入的等待时间,同时也降低吞吐量(因为每一次数据库中的写请求都有额外的开销,比如磁头寻道时间),由于该吞吐量是弹性的,在写入重做日志的数据量较少时对吞吐量是没有要求了,即使吞吐量降低了,也不会影响到按照预设比例进行并行持久化至磁盘的持久化进程,还降低了持久化写入的等待时间(时延);当缓存写满一批重做日志的数据量大时,通过后台进程并行地每次写入单个磁盘的重做日志的数据量也相应增大,吞吐量就更大。

[0134] 本申请一实施例中,如图5所示,基于重做日志中的地址编码与数据库配置,确定需要并行持久化的日志文件为4个,分别为不同的后台进程对应的日志文件1、日志文件2、日志文件3和日志文件4。若缓存中写入了200KB的重做日志,则基于预设比例将写入的这200KB的重做日志拆分成4段。例如预设比例为1:2:3:4,那么所述预设比例1:2:3:4分别对应于待写入重做日志的日志文件1、日志文件2、日志文件3及日志文件4,则有将第一段对应的20KB的重做日志通过后台进程写入日志文件1中,将第二段对应的40KB的重做日志通过后台进程写入日志文件2中,将第三段对应的60KB的重做日志通过后台进程写入日志文件3中,将第四段对应的80KB的重做日志通过后台进程写入日志文件4中,当然将上述四段重做日志写入对应的日志文件时,是并行持久化进行的,实现了将缓存中写入的一批重做日志按照地址编码依序保持递增的顺序分别写入个日志文件。若缓存中写入了100KB的重做日志,则基于预设比例继续将写入的这100KB的重做日志拆分成4段。例如预设比例为1:2:3:4,那么所述预设比例1:2:3:4分别对应于待写入重做日志的日志文件1、日志文件2、日志文件3及日志文件4,则有将第一段对应的10KB的重做日志通过后台进程写入日志文件1中,将第二段对应的20KB的重做日志通过后台进程写入日志文件2中,将第三段对应的30KB的重做日志通过后台进程写入日志文件3中,将第四段对应的40KB的重做日志通过后台进程写入日志文件4中,当然将上述四段重做日志写入对应的日志文件时,是并行持久化进行的,

实现了将缓存中写入的一批重做日志按照地址编码依序保持递增的顺序分别写入个日志文件。

[0135] 为了将每次按照预设比例拆分成段并分别并行持久化写入对应的日志文件中的重做日志进行恢复，则所述写入装置12用于将各段并行写入对应的日志文件的同时，还用于：将各段中的首尾地址编码写入对应的日志文件。例如，由于重做日志的地址编码都是该重做日志的开始地址，故假设缓存中写入了一批重做日志为80KB，按照预设比例(例如1:2:3:4)将该批重做日志进行分段，得到四段重做日志，假设第一段对应的重做日志包括1个重做日，起点地址编码为0，终点地址编码为8KB，第一段对应的重做日志包括2个重做日志，起点地址编码为8KB，终点地址编码为24KB，第三段对应的重做日志包括3个重做日志，起点地址编码为24，终点地址编码为48KB，第四段对应的重做日志包括4个重做日志，起点地址编码为48，终点地址编码为100KB，为了保证后续将持久化写入各个日志文件的重做日志恢复为一份完成有序地重做日志，则在将第一段对应的8KB重做日志写入日志文件1的同时，还将第一段中的首尾地址编码[0,8KB]写入对应的日志文件1中，依此类推，按照预设比例，将各段对应的重做日志分别并行地持久化写入对应的日志文件的同时，还将每段中对应的重做日志的地址编码中的首尾地址编码一并写入对应的日志文件，以便后续基于各段中的首尾地址编码将所有的日志文件中的重做日志恢复出来。

[0136] 为了将每次按照预设比例拆分成段并分别并行持久化写入对应的日志文件中的重做日志进行恢复，则所述写入装置12用于将各段并行写入对应的日志文件的同时，还用于将按预设比例拆分成各段的总数据量写入每个日志文件。例如，由于重做日志的地址编码都是该重做日志的开始地址，故假设缓存中写入了一批重做日志为80KB，按照预设比例(例如1:2:3:4)将该批重做日志进行分段，得到四段重做日志，假设第一段对应的8KB重做日志包括1个重做日，起点地址编码为0，即第一段的总数据量为8KB，第一段对应的重做日志包括2个重做日志，即第二段的总数据量为16KB，第三段对应的重做日志包括3个重做日志，即第三段的总数据量为24KB，第四段对应的重做日志包括4个重做日志，即第四段的总数据量为32KB，为了保证后续将持久化写入各个日志文件的重做日志恢复为一份完成有序地重做日志，则在将第一段对应的8KB重做日志写入日志文件1的同时，还将第一段中的中数据量8KB写入对应的日志文件1中，依此类推，按照预设比例，将各段对应的重做日志分别并行地持久化写入对应的日志文件的同时，还将每段中对应的重做日志的总数据量一并写入对应的日志文件，以便后续基于各段中的首尾地址编码将所有的日志文件中的重做日志恢复出来。

[0137] 例如，在数据库的恢复过程中，按照序号递增的段中的重做日志的地址编码依序保持递增，分别将写入日志文件1中的第一段对应的8KB重做日志，写入日志文件2中的第二段对应16KB重做日志，写入日志文件3中第三段对应的24KB重做日志，写入日志文件4中的第四段对应的32KB重做日志依序地合并，恢复出一份完整地有序地重做日志，实现对持久化至不同磁盘中的日志文件中的重做日志的恢复。

[0138] 为了保证完整地将每一批的需要持久化写入重做日志持久化至对应的日志文件中的，所述写入装置12用于：

[0139] 将各段并行写入对应的日志文件；

[0140] 对完成写入的日志文件个数进行计数；

[0141] 当所述计数的个数等于日志文件的总个数，则判断各段写入对应的日志文件完成。

[0142] 本申请一实施例中，将缓存中写入的每一批需要持久化的重做日志，并行地写入至日志文件时，根据地址编码与数据库中的系统配置，获取当前这一批需要持久化至日志文件的日志文件个数n，当每段重做日志完成写入对应的日志文件之后，需要对完成写入的日志文件个数进行计数并更新计数，当所述计数的个数达到参与并行持久化写入的日志文件的总个数n时，就判断各段重做日志持久化写入至对应的日志文件完成，即完成了将缓存中写入的这一批需要持久化的重做日志的持久化写入。

[0143] 综上所述，本申请通过将缓存中地址编码递增的重做日志按照地址编码依序拆分成段，并将各段并行写入对应的日志文件，其中，日志文件对应与不同的磁盘，实现了将缓存中的重做日志并行写入不同磁盘中对应的日志文件，使得磁盘持久化重做日志时的总的数据量与写入缓存的重做日志的数据量匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题，又由于所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增，以便后续从磁盘中按序将重做日志恢复出来。

[0144] 本申请提供的一种应用于数据库中的将缓存中的重做日志持久化写入至对应的磁盘中的日志文件中，当缓存中写完了一批超市交易记录对应的重做日志256KB，其中，这一批超市交易记录对应的重做日志256KB中的各个重做日志之间按照重做日志的地址编码(例如交易记录的时间先后顺序)依序保持递增，根据数据库中的系统配置以及地址编码，该批重做日志需要并行执行4个后台进程及其分别对应的日志文件1、日志文件2、日志文件3和日志文件4，为了保证减低持久化写入的等待时间，采用按照预设比例(例如1:3:4:8)将缓存中的重做日志进行拆分为4段，并分别通过后台进程对应地持久化写入至各日志文件，则有从地址编码0开始，将第一段对应的16KB重做日志持久化写入至日志文件1，将第二段对应的48KB重做日志持久化写入至日志文件2，将第三段对应的64KB重做日志持久化写入至日志文件3，将第一段对应的128KB重做日志持久化写入至日志文件4，完成多个磁盘中的日志文件并行进行持久化写入缓存中的重做日志，使得磁盘持久化重做日志时的总的数据量与写入缓存的重做日志的数据量匹配得上，进而抹平了磁盘与缓存之间的性能不对等问题，又由于所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增，为了便于后续对重做日志进行恢复，在将每段对应的重做日志写入日志文件的时候，还将每段对应的重做日志的总数据量写入对应的日志文件(例如将第一段对应的16KB重做日志持久化写入至日志文件1的同时，将第一段对应的重做日志的总数据量16KB写入日志文件1)，以便后续从磁盘中按序将重做日志恢复出来。

[0145] 根据本申请的另一个方面，提供了一种基于计算的设备，其中，该设备包括：

[0146] 处理器；以及

[0147] 被安排成存储计算机可执行指令的存储器，所述可执行指令在被执行时使所述处理器：

[0148] 将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

[0149] 将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

[0150] 根据本申请的另一个方面，提供了一种存储可执行指令的非暂态计算机可读存储介质，在所述可执行指令由电子设备执行时，使得所述电子设备：

[0151] 将缓存中地址编码递增的重做日志按地址编码依序拆分成段，其中，所述段中的重做日志的地址编码依序保持递增，序号递增的段中的重做日志的地址编码依序保持递增；

[0152] 将各段并行写入对应的日志文件，其中，所述日志文件对应于不同的磁盘。

[0153] 显然，本领域的技术人员可以对本申请进行各种改动和变型而不脱离本申请的精神和范围。这样，倘若本申请的这些修改和变型属于本申请权利要求及其等同技术的范围之内，则本申请也意图包含这些改动和变型在内。

[0154] 需要注意的是，本申请可在软件和/或软件与硬件的组合体中被实施，例如，可采用专用集成电路(ASIC)、通用目的计算机或任何其他类似硬件设备来实现。在一个实施例中，本申请的软件程序可以通过处理器执行以实现上文所述步骤或功能。同样地，本申请的软件程序(包括相关的数据结构)可以被存储到计算机可读记录介质中，例如，RAM存储器，磁或光驱动器或软磁盘及类似设备。另外，本申请的一些步骤或功能可采用硬件来实现，例如，作为与处理器配合从而执行各个步骤或功能的电路。

[0155] 另外，本申请的一部分可被应用为计算机程序产品，例如计算机程序指令，当其被计算机执行时，通过该计算机的操作，可以调用或提供根据本申请的方法和/或技术方案。而调用本申请的方法的程序指令，可能被存储在固定的或可移动的记录介质中，和/或通过广播或其他信号承载媒体中的数据流而被传输，和/或被存储在根据所述程序指令运行的计算机设备的工作存储器中。在此，根据本申请的一个实施例包括一个装置，该装置包括用于存储计算机程序指令的存储器和用于执行程序指令的处理器，其中，当该计算机程序指令被该处理器执行时，触发该装置运行基于前述根据本申请的多个实施例的方法和/或技术方案。

[0156] 对于本领域技术人员而言，显然本申请不限于上述示范性实施例的细节，而且在不背离本申请的精神或基本特征的情况下，能够以其他的具体形式实现本申请。因此，无论从哪一点来看，均应将实施例看作是示范性的，而且是非限制性的，本申请的范围由所附权利要求而不是上述说明限定，因此旨在将落在权利要求的等同要件的含义和范围内的所有变化涵括在本申请内。不应将权利要求中的任何附图标记视为限制所涉及的权利要求。此外，显然“包括”一词不排除其他单元或步骤，单数不排除复数。装置权利要求中陈述的多个单元或装置也可以由一个单元或装置通过软件或者硬件来实现。第一，第二等词语用来表示名称，而并不表示任何特定的顺序。


redo buffer
缓存区
30GB/s
持久化
redo file
单块磁盘
2GB/s

图1

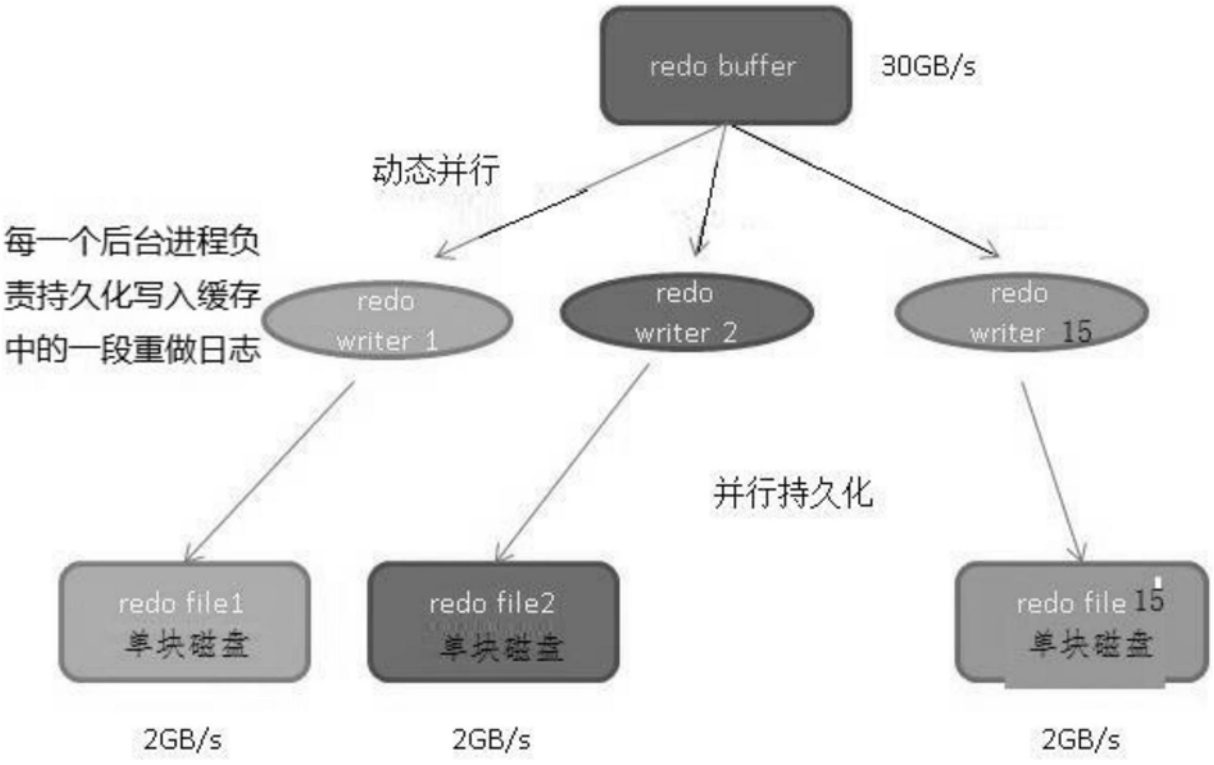

redo buffer
30GB/s
动态并行
每一个后台进程负
责持久化写入缓存
中的一段重做日志
redo
writer 1
redo
writer 2
redo
writer 15
并行持久化
redo file1
单块磁盘
redo file2
单块磁盘
redo file 15
单块磁盘
2GB/s
2GB/s
2GB/s

图2

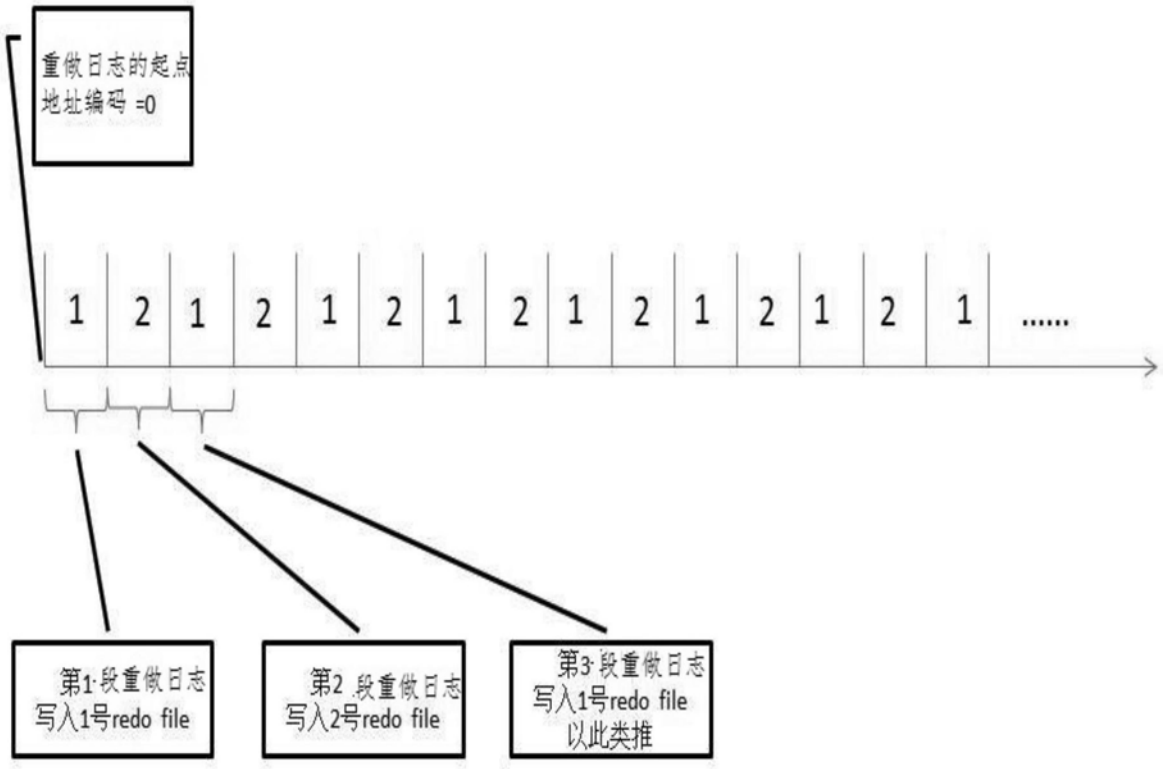

重做日志的起点
地址编码 =0
1 2 1 2 1 2 1 2 1 2 1 2 1 2 1 .....
第1段重做日志
写入1号redo file
第2段重做日志
写入2号redo file
第3段重做日志
写入1号redo file
以此类推

图3

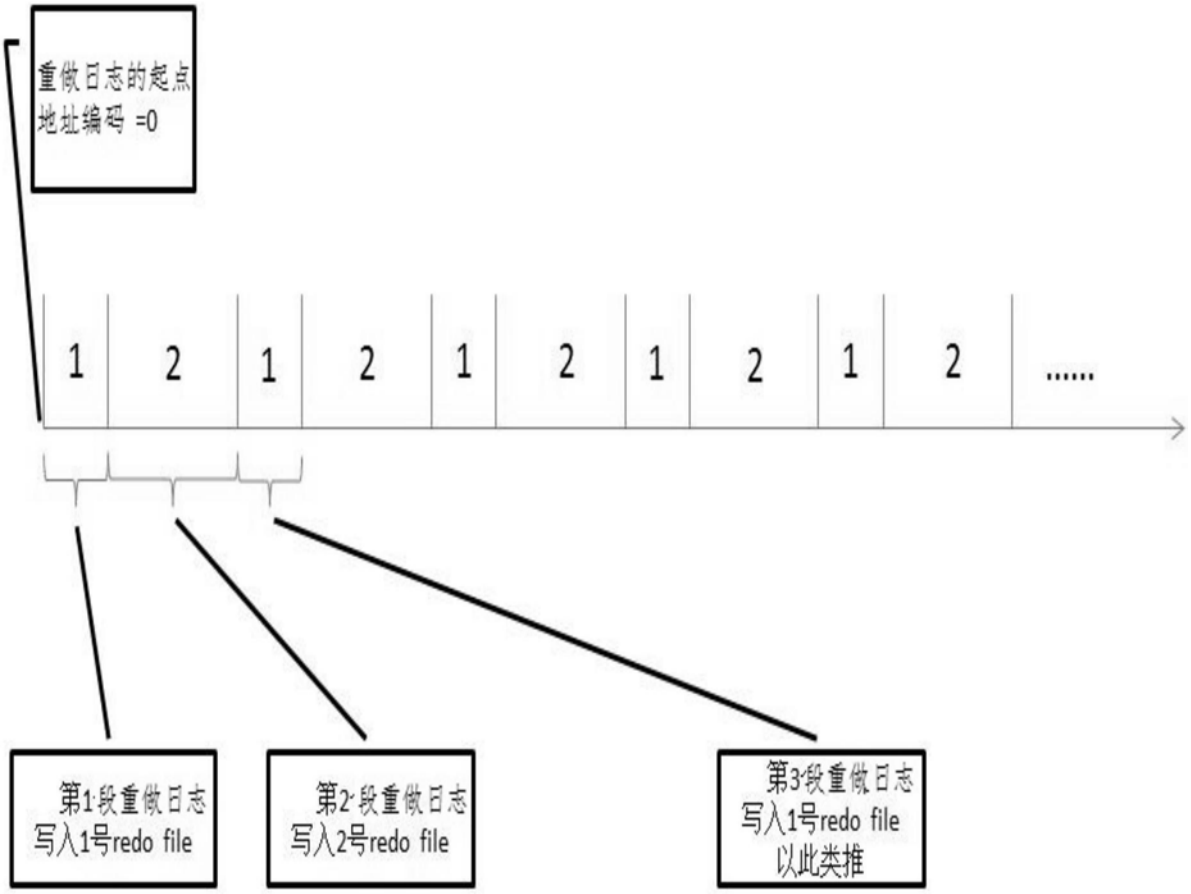

重做日志的起点
地址编码 =0
1
2
1
2
1
2
1
2
1
2
.....
第1段重做日志
写入1号redo file
第2段重做日志
写入2号redo file
第3段重做日志
写入1号redo file
以此类推

图4

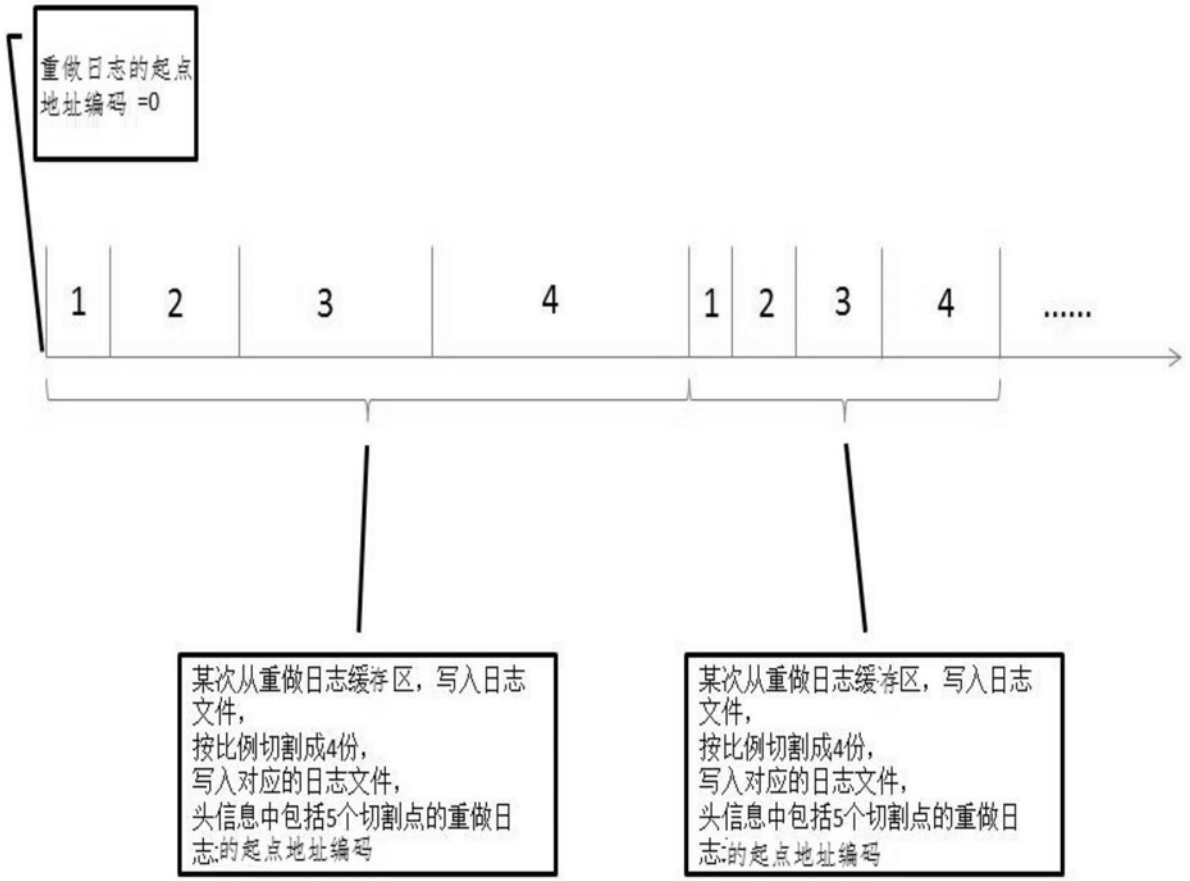

重做日志的起点地址编码 =0
1
2
3
4
1
2
3
4
.....
某次从重做日志缓存区，写入日志文件，
按比例切割成4份，
写入对应的日志文件，
头信息中包括5个切割点的重做日志的起点地址编码
某次从重做日志缓存区，写入日志文件，
按比例切割成4份，
写入对应的日志文件，
头信息中包括5个切割点的重做日志的起点地址编码

图5

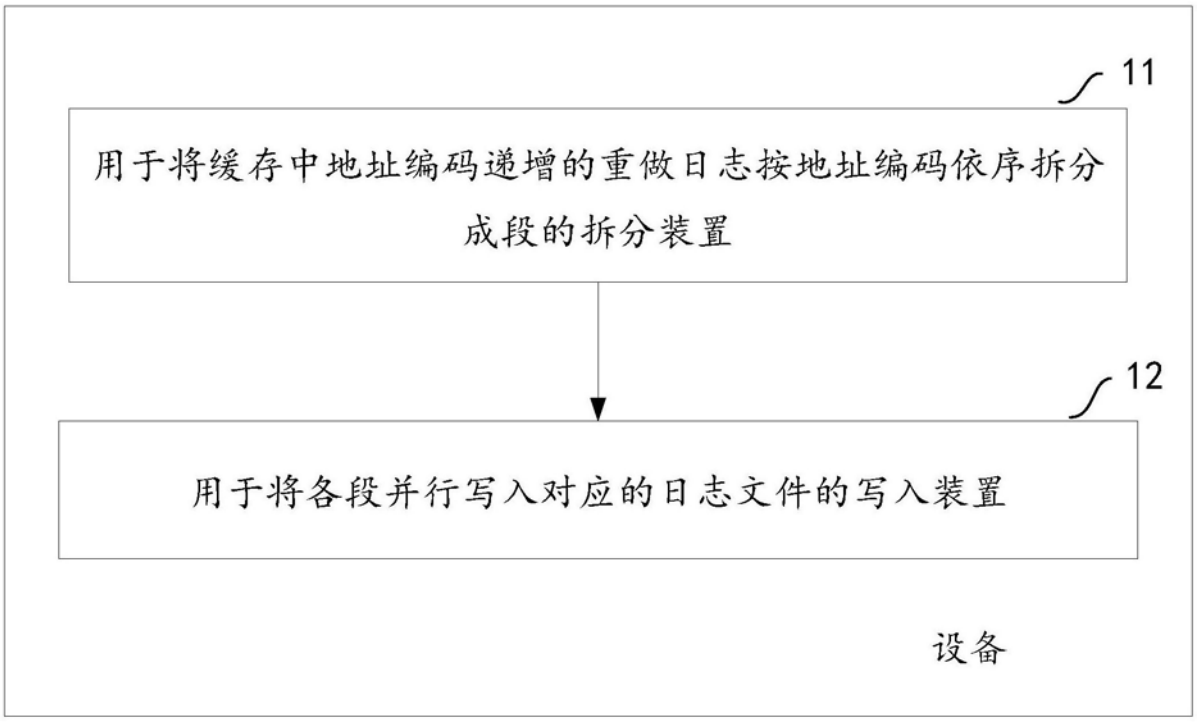

11
用于将缓存中地址编码递增的重做日志按地址编码依序拆分成段的拆分装置
12
用于将各段并行写入对应的日志文件的写入装置
设备

图6